Syntactic Cues or Semantic Constraints? A Split That Demystifies LLM Type Inference and Nondeterminism

ANONYMOUS AUTHOR(S)

Abstract...

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper;

Additional Key Words and Phrases: Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Anonymous Author(s). 2026. Syntactic Cues or Semantic Constraints? A Split That Demystifies LLM Type Inference and Nondeterminism. 1, 1 (September 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Large language models (LLMs) are remarkably capable, yet their predictions can be surprisingly unstable. Repeating the same request can produce different answers, even when the input and model configuration are unchanged. Ouyang et al. report substantial non-determinism in LLM-based code generation and show that setting the temperature to zero does not guarantee deterministic outputs [3]. More broadly, Atil et al. observe accuracy differences of up to 10% across identical runs of deterministically configured LLMs [2]. LLMs can also be highly sensitive to information that should not affect the underlying task: Sclar et al. report large performance differences under meaning-preserving changes to prompt formatting [4]. These studies establish an increasingly important practical problem: the same LLM can behave differently even when the underlying semantic task remains unchanged.

What remains less understood is *why this instability is selective*. Within the same program and under the same model, some individual predictions remain unchanged across repeated generations, whereas others fluctuate substantially. Prior studies primarily quantify nondeterminism across tasks, prompts, models, or aggregate benchmark scores. They provide considerably less explanation for a more fine-grained question: *what makes one individual inference stable while another is not?* A related question is why some predictions survive meaning-preserving perturbations while others lose substantial information. Answering these questions is important for the reliability of LLM-assisted software engineering. Aggregate accuracy from a single run can hide substantial uncertainty if different predictions are supported by qualitatively different amounts of semantic evidence.

We investigate these questions through *LLM-based type inference for Python*. Type inference offers an unusually useful setting for studying the evidence behind an LLM prediction. Some type information is imposed by the program itself. Function calls relate actual arguments to formal parameters; assignments propagate types between expressions; returns constrain function results; constructors connect arguments with object state; and attributes, containers, and dependency

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
XXXX-XXXX/2026/9-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

signatures introduce additional typing relationships. Other type information is not uniquely determined by these relationships. A model may nevertheless predict a precise project-specific type by exploiting identifier names, API names, naming conventions, or associations learned during training. The usefulness of both program structure and lexical information is well established in learned type inference. For example, Typilus probabilistically reasons over program structure, names, and patterns [1], while HiTyper shows that static constraints and learned predictions provide complementary benefits [**?**]. We ask a different question: *can the degree to which program semantics constrain a type explain when an LLM depends on lexical cues and when its predictions become unstable?*

We call these two forms of evidence *semantic constraints* and *syntactic cues*. By semantic constraints, we mean typing relationships induced by the meaning of program constructs—including calls, assignments, returns, constructors, attributes, containers, and dependency signatures—whose relevant relationships are preserved under an isomorphic renaming of project identifiers. By syntactic cues, we mean lexical information carried by identifiers, API and type names, and related naming conventions. Our terminology is operational: isomorphic renaming removes or substantially weakens project-specific syntactic cues while preserving the semantic relationships from which typing constraints arise.

The central observation of this paper is that this distinction is not binary. An annotation need not be either "constraint-based" or "name-based." Instead, different annotation positions—and even different components of the same structured type—can be constrained to different degrees. Consider a program for which the outer type `dict` and key type `str` are determined by program operations, while the value type remains unresolved. The semantic information available to an inference procedure is then better represented as

`dict[str,?]`

than as either "constrained" or "unconstrained." A model predicting `dict[str,User]` has preserved the semantically determined structure while supplying `User` from evidence outside those constraints. If isomorphic renaming changes the prediction to `dict[str,Any]`, the model has not forgotten the entire type: it has lost only the residual specificity that the program did not determine.

A motivating example illustrates why this distinction cannot be made by inspecting a predicted type alone. In one program in our corpus, GPT-5 produces annotations corresponding to

```
def inject_advertisement(
    hass: Any,
    device: BLEDevice,
    adv: AdvertisementData,
) -> None:
    ...
```

At first glance, the project-specific types `BLEDevice` and `AdvertisementData` appear to follow from lexical cues such as `device`, `adv`, and `inject_advertisement`. Tracing the program, however, reveals typing relationships elsewhere in the target and its supplied dependency context that propagate these types to the annotation site. What appears to be a name-based prediction therefore has a semantic-constraint explanation. In other cases, only a shell such as `list[?]`, `dict[str,?]`, or `tuple[?,T]` can be established, leaving substantive components for the model to complete.

To make this distinction explicit, we develop a conservative *semantic inferability analysis*. Given exactly the unannotated target program and dependency context supplied to the LLM, the analysis derives typing constraints for annotation positions and tracks their propagation through the program. Crucially, we characterize the *completeness* of the resulting semantic information rather

than merely asking whether some constraint exists. A *concrete* constraint contains no unresolved structural holes and largely determines the type, such as `str` or `dict[str,str]`. A *partial* constraint determines substantive nested components while leaving others unresolved, such as `dict[str,?]`. A *shape-only* constraint determines primarily an outer structure such as `list[?]` or `dict[?,?]`. Finally, *none* indicates that our analysis derives no useful structural type reference. For structured annotations, this representation additionally identifies which components are fixed and which remain open.

We use this notion of constrainedness as an explanatory variable in a controlled study of 497 Python files. Our matched multi-run analysis contains 52,596 annotation sites, of which 49,716 are present in both the original-name and isomorphically renamed conditions. We generate annotations four times with the original programs and dependency contexts and four times after applying an isomorphic renaming. Renaming preserves the program relationships from which semantic constraints arise while removing project-specific lexical identities; repeated generations within each condition preserve both the semantic constraints and the available cues. This design lets us study two phenomena through the same semantic decomposition: *information loss when syntactic cues are removed* and *run-to-run instability when the available evidence is held fixed.*

The renaming experiment shows that removal of syntactic cues produces a strongly asymmetric loss of type information. Across sites whose modal predictions are comparable, 8,293 become less specific after renaming, whereas only 209 become more specific. The loss is often component-wise: predictions such as `dict[str,str]` become `dict[str,Any]`, or `list[Name]` becomes `list[Any]`, while the semantically constrained outer structure is preserved. More broadly, removing lexical identities shifts the model toward low-information completion. Pooled across the four runs, the fraction of annotation sites containing `Any` rises from 16.08% with original names to 34.46% after isomorphic renaming, while the fraction containing `object` rises from 2.82% to 4.66%. Thus, syntactic cues are not merely selecting among synonymous spellings: at positions not fully determined by semantic constraints, they often provide information that allows the model to produce a more specific completion.

The same constrainedness classification also explains run-to-run stability. A simple constrained-versus-unconstrained split is insufficient. Across four original-name generations, complete four-run stability is 82.7% for concrete sites, 79.5% for sites with no useful semantic constraint, 67.2% for shape-only sites, and only 56.0% for partial sites. After isomorphic renaming, the corresponding rates are 78.1%, 75.1%, 63.7%, and 55.6%, respectively. Both conditions therefore exhibit the same ordering:

$$\textit{concrete} > \textit{none} > \textit{shape-only} > \textit{partial}.$$

The least stable predictions are not those for which semantic information is completely absent. They are those at which semantic constraints determine *part* of the answer while leaving substantive degrees of freedom unresolved.

The behavior of unconstrained sites helps explain this initially counterintuitive result. With original identifiers, 85.4% of stable unconstrained sites nevertheless receive stable specific types, while only 12.0% receive a low-information root such as `Any` or `object`. After renaming, stable specific predictions fall to 63.5%, while stable low-information roots rise to 32.2%. Thus, a site can be semantically unconstrained yet stable because the model consistently follows a strong syntactic or learned prior; when those cues are weakened, the model can instead settle consistently on a low-information fallback. Partial and shape-only sites occupy a more difficult middle ground: semantic constraints restrict the space of possible answers without uniquely determining the remaining components, leaving multiple plausible completions available.

These observations suggest a behavioral model of LLM type inference that we call *constraint-guided completion*. Semantic constraints define an invariant core and restrict the admissible solution space; syntactic cues and learned priors supply the residual information needed to complete unresolved positions. When constraints are concrete, little completion is necessary and predictions are comparatively robust. When only the outer shape or some nested components are known, the model must choose among competing completions, increasing both cue dependence and run-to-run variability. When no useful semantic constraint exists, predictions can still be stable because a strong prior or low-information fallback can dominate consistently. Constraint *completeness*, rather than constraint presence alone, therefore provides a common mechanism for explaining both information loss under renaming and nondeterminism across repeated generations.

Finally, we ask whether the information lost under renaming has observable consequences for the generated program. We transplant the LLM-generated annotations into the full implementations and type-check them with `mypy` in project-specific dependency environments. On the matched baseline-clean subset, exact renamed outputs introduce 9.042 unique target diagnostics per 100 LLM annotation sites, compared with 8.345 for original-name outputs, an 8.35% increase. However, this difference is strongly affected by how the model represents low-information uncertainty. In a sensitivity analysis that replaces LLM-produced `object` with `Any`, the diagnostic burden falls to 6.922 per 100 sites for original outputs and 6.240 for renamed outputs. The normalization therefore reduces diagnostic burden much more strongly for renamed predictions, consistent with their increased use of `object`.

Frequency alone, however, does not determine the downstream effect. Among 556 baseline-clean renamed records containing at least one `object`, replacing `object` with `Any` changes no introduced diagnostic in 40.8% of cases. The median reduction is only one diagnostic, yet the 90th percentile is 20 and the maximum is 225. Manual inspection confirms the mechanism: a small number of `object` annotations at highly exposed positions, such as widely used module-valued globals, can trigger large cascades of attribute and call errors, whereas the same number of `object` annotations flowing through weakly typed interfaces can have almost no effect. The downstream cost of lost specificity therefore depends not only on *how much* information is lost, but also on *where* that information participates in the program's typing relationships.

Taken together, these results provide three complementary views of the same mechanism. Semantic inferability identifies how much of each annotation is fixed by the program. Isomorphic renaming reveals which remaining information depends on syntactic cues. Repeated generation reveals how stable the resulting completion is, while project-context type checking reveals when the lost specificity becomes operationally consequential. Rather than treating LLM type inference as either semantic reasoning or lexical pattern matching, our results show that the two operate compositionally: semantic constraints delimit the answer, while syntactic cues and learned priors complete what those constraints leave unresolved.

This paper makes the following contributions:

- **A fine-grained semantic-constrainedness model for LLM type inference.** We characterize annotation positions and components by how completely they are determined by program semantics—concrete, partial, shape-only, or unconstrained—and develop a semantic inferability analysis that derives these constraints and records their provenance and propagation. This moves beyond the common binary distinction between "structurally inferable" and "name-based" predictions.
- **Constrainedness as an explanation for type-information loss under lexical perturbation.** Using isomorphic renaming, we show that semantic constraints provide an invariant core while unresolved components depend strongly on syntactic cues. Across

comparable modal predictions, 8,293 sites become less specific after renaming compared with only 209 that become more specific; `Any` usage rises from 16.08% to 34.46%, and `object` usage from 2.82% to 4.66%. The losses frequently occur within unresolved components of otherwise preserved type structure.

- **Constraint completeness as a mechanism for LLM nondeterminism.** Across four independent original and four renamed generations, stability follows the same ordering—concrete, none, shape-only, partial—with original-name four-run stability of 82.7%, 79.5%, 67.2%, and 56.0%, respectively, and renamed stability of 78.1%, 75.1%, 63.7%, and 55.6%. These results show that nondeterminism is governed more strongly by how completely semantics determine an answer than by whether any semantic constraint is present.
- **A downstream consistency analysis of cue-dependent type loss.** Project-context `mypy` evaluation shows an 8.35% increase in introduced diagnostic density for exact renamed outputs on baseline-clean files. An `object`-to-`Any` sensitivity analysis reveals that part of this burden comes from the representation and placement of low-information predictions: 40.8% of baseline-clean object-containing records show no diagnostic effect, while a small number produce cascades of up to 225 diagnostics.

These findings suggest that LLM type inference is best understood as *constraint-guided completion*. Semantic constraints determine an invariant portion of the answer; syntactic cues and learned priors supply residual specificity where those constraints are incomplete. The degree of semantic constrainedness consequently explains both which type information survives lexical perturbation and which predictions remain stable across repeated generations, while the program position of the unresolved information determines whether its loss becomes consequential to static checking.

DATA AVAILABILITY

The data package, including prompts, evaluated programs, LLM outputs, and other scripts, is available in the following link https://anonymous.4open.science/r/Harmonizing_ml_results-EC8C/README.md. We will submit the package for artifact evaluation upon acceptance.

REFERENCES

[1] Miltiadis Allamanis, Earl T. Barr, Soline Ducousso, and Zheng Gao. 2020. Typilus: Neural Type Hints. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 91–105. https://doi.org/10.1145/3385412.3385997

[2] Berk Atil, Alexa Chittams, Liseng Fu, Ferhan Ture, Lixinyu Xu, and Breck Baldwin. 2024. LLM Stability: A detailed analysis with some surprises. *arXiv preprint arXiv:2408.04667* 1 (2024).

[3] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. 2025. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–28.

[4] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, Vol. 2024. 25055–25083.